

Architecture Status Report

ExamForge AI | Generated: 2026-08-02

Technology Stack

Layer	Technology	Version	Status
Framework	Next.js	16.1.1	Active, configured
UI Library	React	19.0.0	Active, configured
Auth	Supabase Auth (SSR)	2.111.0	Active, configured
Database	Supabase (PostgreSQL)	N/A	Active, live queries
ORM	Prisma	6.11.1	DEAD - installed but never used
State	Zustand	5.0.14	Partial - 3/5 stores unused
Query	TanStack React Query	5.101.4	DEAD - configured but never used
Offline	Dexie	4.4.4	DEAD - installed but never used
Charts	Recharts	3.10.1	Active, not dynamically imported
UI Components	shadcn/ui (Radix)	Various	Active, 40+ components
Styling	Tailwind CSS	4.x	Active, configured
Animations	Framer Motion	12.43.0	Active, in main bundle
Toasts	Sonner	2.0.7	Active, configured
Forms	React Hook Form + Zod	7.84.0 / 4.4.3	Active, configured
PWA	next-pwa	5.6.0	DEAD - installed but not configured
i18n	next-intl	4.3.4	DEAD - installed but not configured

Architecture Components

The application follows a standard Next.js App Router architecture with route groups for (public) and (app) layouts. Server Components are used for data fetching on most pages, though the app layout and several key pages (analytics, notifications, settings) are client components. The middleware implements a 3-layer guard chain (AuthGuard, OnboardingGuard, RoleBasedGuard), though the role-based guard is currently non-functional due to route prefix mismatch.

Active Architecture Paths

The following architecture paths are actively used and functional: Supabase SSR for authentication, Server Components for data fetching on most pages, Server Actions for mutations (create school, mark notification read, delete notification), Realtime subscriptions via Supabase channels, Zustand for auth and notification state, and the shadcn/ui component library for UI elements.

Dead Architecture Paths

The following architecture paths are installed but never used: Prisma ORM (db.ts is never imported), TanStack React Query (QueryProvider mounted but no useQuery calls), Dexie offline storage (never configured), next-pwa (no PWA config in next.config.ts), next-intl (no i18n

configuration), next-auth (Supabase Auth is used instead), the API client module (api/client.ts, errors.ts, result.ts), the toast service wrapper (toast-service.ts), and three Zustand stores (ui-store, exam-session-store, theme-store preferences).

Data Flow Architecture

Server Component Pages: Page.tsx (Server Component) calls createClient() from supabase/server.ts, fetches data via supabase.from().select(), renders with RSC, and passes data as props to client components. This is the correct pattern and is used by: schools, students, teachers, parents, results, CBT, question-bank, marketplace, billing, dashboard, and reports pages.

Client Component Pages: Analytics, notifications, and settings pages use "use client" with useEffect + supabase client for data fetching. This is the incorrect pattern for initial data loads and should be refactored to use Server Components for initial data.

Server Actions: Mutations use Server Actions defined in features/ directory. These call revalidatePath() for cache invalidation. The pattern is correct but lacks authorization checks.

Realtime: RealtimeProvider in the app layout subscribes to notifications and updates the NotificationStore. The notifications page also subscribes independently, creating duplicate subscriptions. This should be consolidated.

Infrastructure Dependencies

Service	Provider	Status	Notes
Authentication	Supabase Auth	Active	SSR pattern, cookie-based sessions
Database	Supabase PostgreSQL	Active	RLS policies assumed but not verified from code
Realtime	Supabase Realtime	Active	WebSocket subscriptions, duplicate channels
Storage	Supabase Storage	Active	Configured for avatar images
Edge Functions	Supabase Edge Functions	Active	AI, billing, CBT timing, marketplace, search
Payments	Flutterwave	Active	Webhook proxy via Next.js API route
Hosting	Next.js Standalone	Configured	output: "standalone" in next.config.ts
CDN	Not configured	MISSING	No CDN for static assets
Monitoring	Not configured	MISSING	No Sentry, OpenTelemetry, or analytics
Rate Limiting	Supabase Edge Functions only	Partial	No rate limiting on Next.js API routes

Architecture Recommendations

1. Remove dead architecture paths (Prisma, TanStack Query if not adopting, Dexie if not implementing offline, next-pwa, next-intl, next-auth) to reduce bundle size and maintenance burden.
2. Adopt TanStack Query for client-side data fetching in analytics and notifications pages.
3. Implement Dexie for offline support in CBT exam sessions.
4. Add monitoring infrastructure (Sentry for errors, OpenTelemetry for tracing).
5. Add rate limiting at the Next.js API route level.
6. Configure Content Security Policy headers.
7. Consolidate realtime subscriptions to use a single channel per table.